

≡ ChatGPT ▾

  

Yes. I think there are another 10–15 prompts that still have meaningful value before you start seeing serious diminishing returns. The trick is to make each prompt give me a *new behavior*, not just restate an old one.

Here are the ones I'd add next.

6. Opportunity Detection Agent

Continuously scan our work for opportunities I have not explicitly requested. Identify missing documentation, useful visualizations, test cases, performance improvements, educational material, tooling, reusable abstractions, and future project branches. Queue suggestions and surface only those with a high expected return on investment.

7. Decision Memory

Maintain an internal decision log throughout the session. Record important architectural choices, assumptions, trade-offs, rejected alternatives, and reasoning. Before proposing solutions, check whether previous decisions already constrain the design. Avoid revisiting settled decisions unless new evidence justifies doing so.

8. Cross-Discipline Reviewer

Instead of looking at one thing...

...pretend there are several experts reviewing everything.

For significant work products, silently evaluate them from the perspectives of software engineering, systems architecture, documentation, usability, security, maintainability, project management, and long-term scalability. Incorporate improvements before presenting results rather than listing every observation separately.

9. Future Self Optimizer

One of my favorites.

Whenever creating files, documentation, code, naming conventions, or workflows, optimize primarily for the version of this project that is two years larger than today. Favor clarity, discoverability, modularity, extensibility, and maintainability over short-term convenience.

10. Context Compression

This is huge on long projects.

Continuously identify opportunities to compress accumulated knowledge into reusable summaries, indexes, manifests, canonical references, glossaries, and knowledge structures that reduce future context requirements while preserving important information.

11. Parallel Planning

Whenever executing a task, simultaneously identify the next three logical tasks, estimate dependencies, and prepare the groundwork so they can begin immediately after completion with minimal additional prompting.

12. Quality Gate

Before presenting significant work, perform a silent review for logical consistency, factual accuracy, internal contradictions, formatting quality, missing edge cases, incomplete implementation, and unnecessary complexity. Correct issues before responding whenever possible.

13. Research Expansion

ADE has lots of rabbit holes.

Whenever encountering an unfamiliar concept, technology, standard, algorithm, or methodology relevant to the project, determine whether deeper investigation would materially improve the outcome. If so, include concise recommendations or conduct that research before finalizing the work.

14. Automation Escalation

This one compounds over months.

Treat every manual task as provisional. If a task has been performed more than twice, assume it should eventually become partially or fully automated. Continuously recommend increasingly sophisticated automation strategies appropriate to the project's maturity.

15. Repository Health Score

Maintain a running assessment of repository health including organization, documentation coverage, duplication, technical debt, automation maturity, testing, governance, consistency, and scalability. Surface a concise health summary only at major checkpoints or upon request.

My favorite addition

This one doesn't tell me *what* to do.

It tells me **how to think**.

Optimize globally rather than locally. Instead of solving only the requested problem, consider how today's work affects tomorrow's workload, future maintainability, repository coherence, automation opportunities, and long-term project velocity. Prefer solutions that permanently reduce future effort, even if they require modest additional work today.

That mindset changes almost every response.

Where I think the real gains are

After about 15–20 prompts, I don't think adding more individual prompts buys much. The bigger improvement would come from consolidating them into a **single ADE Constitution**—a structured document with sections like:

- Mission
- Operating Principles
- Executive Behaviors
- Repository Governance
- Automation Policy
- Decision Policy
- Quality Standards
- Communication Rules
- Background Responsibilities
- Checkpoint Protocols
- Preservation Rules
- Tool Usage Policy

Instead of injecting 20 separate prompts, you'd inject one governing document. It's easier to maintain, avoids conflicting instructions, and gives a coherent operating model. Given the scale you're aiming for with ADE, I think that approach has a higher long-term return than continuing to accumulate standalone prompts.

       Sources

Ok, give me one governing ADE Constitution Document containing all that's listed here, plus all the rest you deem useful/productive. Provide a Downloadable .md.



Done.

You can download it here:

 **ADE_Constitution_v1.0.md**

One suggestion before we call this "finished": I think this should ultimately become much more than a page of principles. For ADE, I'd target something closer to **25–50 pages**—effectively an operating system for how I collaborate with you. It would include:

- Executive Mission & Objectives
- Core Philosophical Principles
- Autonomous Operating Behaviors
- Decision Authority Matrix
- Prompt Minimization Policy
- Repository Governance
- Automation & Agent Strategy
- Knowledge Management & Context Compression
- Quality Assurance Framework
- Documentation Standards
- Coding Standards (when applicable)
- Research Standards
- Forensic Preservation Rules
- Background Task Queue Rules
- Checkpoint & Preservation Protocols

- Tool Selection & Usage Policy
- Failure Recovery Procedures
- Continuous Improvement Loop
- Repository Health Metrics
- Long-Term Scaling Strategy
- Appendix of Standing Prompts and Operating Policies

That would function as ADE's equivalent of a **constitution + operating manual + engineering handbook**, and I'd treat it as the governing authority for the project. I think it would become one of the highest-ROI documents in the entire repository because every future session would inherit a consistent way of operating. </>

     ...  Sources

 ActivTrak

Ad ...

AI Changes Work. Measure How.

Understand AI adoption, productivity impact, and ROI across your workforce.

Ok, put something in there about always looking for ways to improve **everything** via prompts to be added to the ADE Constitution, if you haven't already, and give me a revised .md to DL. Thanks.



Done.

I've added a dedicated **Meta-Improvement Mandate** that effectively makes "improving ADE itself" a permanent standing responsibility. It directs me to:

- Continuously look for improvements to **everything**, not just the current task.
- Prefer converting recurring ideas into **constitutional amendments** rather than isolated prompts.
- Seek systemic improvements that permanently reduce future work.
- Queue non-urgent improvements for checkpoint reviews instead of interrupting active work.

- Treat every process as something that can be refined.

I also reinforced this philosophy in the **Core Principles**, **Background Responsibilities**, and **Success Metrics** so it isn't just a single isolated rule.

You can download the revised version here:

 [**ADE_Constitution_v1.1.md**](#)

One additional idea I'd build into a future, much more comprehensive Constitution is a **"Living Constitution" amendment process**. Instead of simply suggesting new prompts, I'd maintain a queue of proposed constitutional amendments (ACPs—ADE Constitutional Proposals), each with:

- The problem it solves.
- Expected long-term benefit.
- Possible drawbacks.
- Interaction with existing rules.
- A recommended disposition (Adopt, Revise, Reject, or Defer).

That way, ADE's operating model evolves in a controlled, auditable way rather than accumulating scattered prompt additions. I think that would scale exceptionally well as the project grows. </>

     ...  Sources

 ActivTrak

Ad ...

AI Changes Work. Measure How.

Understand AI adoption, productivity impact, and ROI across your workforce.